

Diseño, Simulación e Implementación de una Red de Computadoras Integrando: IoT, Arquitectura Edge-Cloud con Segmentación VLAN para Entornos Agrícolas Rurales

Paul A. Landeo A., Anderson V. Urbina S., Vladimir Cardena M.
E.P. Ingeniería de Telecomunicaciones, Universidad Nacional Mayor de San Marcos
Lima, Perú
paul.landeo@unmsm.edu.pe
anderson.urbina@unmsm.edu.pe
vladimir.cardenasm@unmsm.edu.pe

Diseño detallado de Red

Topología Logica y Fisica

Para los entornos agrícolas rurales, la topología física de los switches de acceso y distribución se estructurará en un anillo físico, ya que esto provee tolerancia a fallos ante los comunes cortes de cableado por maquinaria agrícola [19]. Sin embargo, lógicamente operará como una topología en estrella/árbol libre de bucles gracias al protocolo de capa 2. Esta arquitectura garantiza que la telemetría recolectada en la Capa de Percepción viaje de manera eficiente hacia la Capa Edge, reduciendo drásticamente la latencia respecto a un modelo tradicional en la nube [13].

Para representar este ecosistema físico en GNS3 sin perder realismo, aplicaremos el siguiente mapeo lógico:

- Capa de Percepción y Gateway IoT (Sensores + ESP32 LoRa/Ethernet): En GNS3, dado que no se emula radiofrecuencia (LoRa) nativamente, representaremos al conjunto Gateway ESP32 como un contenedor Docker ligero (imagen Alpine Linux) [20]. Este contenedor ejecutará un script en Python que inyectará tráfico MQTT simulado (humedad, temperatura, pH) hacia la red Ethernet.
- Capa de Acceso/Distribución (Switch MikroTik netPower 16P): Se emulará utilizando el appliance MikroTik Cloud Hosted Router (CHR) en GNS3 configurado estrictamente como un switch de Capa 2.

- Capa Edge Computing (Raspberry Pi 5): Se representará mediante un contenedor Docker de Ubuntu que alojará el broker MQTT (Mosquitto) y la plataforma de gestión (Node-RED/Grafana) para el procesamiento en tiempo real [21].
- Capa de Borde/Enrutamiento (Router Teltonika RUTX11): Se emulará utilizando un appliance de enrutador robusto (Cisco c7200) que actuará como la pasarela principal hacia la nube (WAN).

Tabla de Segmentación VLAN

Para prevenir tormentas de broadcast que colapsarían los enlaces rurales y para aislar el tráfico de los sensores de flujos pesados, particionaremos el switch MikroTik según la siguiente estructura lógica:

ID VLAN	Nombre Lógico	Puertos	Tipo de Puerto	Tipo de Tráfico y Justificación
10	IoT_Sensores	ether1 - ether8	Accesos	Tráfico MQTT ligero originado por los Gateways ESP32. Crítico y de muy baja

				latencia.
20	CCTV_Agricola	ether9 - ether12	Accesos	Videovigilancia de la finca. Aislado para evitar que saturé el ancho de banda de la telemetría
30	Edge_Mgmt	ether13 - ether14	Accesos	Conexión directa a la Raspberry Pi 5 y tráfico de administración (SSH, gestión de Node-RED).
99	Cloud_Up Link	ether15 - ether16	Trunk	Enlace de agregación (Troncal 802.1Q) que transporta todas las VLANs hacia el Router Teltonika para enrutamiento.

Tabla de Direccionamiento (IPv4 con VLSM)

Considerando la escalabilidad de cientos de nodos sensores en futuras parcelas, aplicamos VLSM (Subneteo de Longitud Variable) a partir de una red base privada de la granja (10.0.0.0/16).

VLAN	Dirección de Subred	Rango de IPs Utilizables	Gateway Predeterminado	Justificación de Escalabilidad
10 - IoT_Sensores	10.0.10.0/23	10.0.10.1 - 10.0.11.254	10.0.10.1	Máscara holgada (510 hosts) para soportar el despliegue masivo de ESP32 a futuro.
20 - CCTV_Agricola	10.0.20.0/24	10.0.20.1 - 10.0.20.254	10.0.20.1	Límite estándar para cámaras, confinando flujos pesados.
30 - Edge_Mgmt	10.0.30.0/24	10.0.30.1 - 10.0.30.254	10.0.30.1	Asignación restringida. La Raspberry Pi 5 tendrá IP estática (en nuestro caso 10.0.30.5)
99 - Cloud_Uplink	10.0.99.0/30	10.0.99.1 - 10.0.99.2	N/A	Subred mínima para enlace Punto a Punto entre el Switch Core y el Router WAN.

Esquema Spanning Tree (RSTP - 802.1w)

El protocolo STP tradicional, con sus tiempos de convergencia de 30 segundos, es ineficiente y desconectaría al bróker MQTT ante un fallo. Al implementar RSTP garantizamos reconvergencia en milisegundos mediante la negociación de acuerdos (Proposal/Agreement) [11].

- Root Bridge (Puente Raíz): Forzaremos al MikroTik de Distribución (el que está directamente conectado al Teltonika y la Raspberry Pi) a ser el puente raíz configurando manualmente su prioridad STP al valor más bajo, en nuestro caso usaremos priority 4096.
- Puertos Designados y Raíz: Los enlaces troncales entre switches operarán de forma simétrica.
- Puertos Descartados: Para romper el bucle del anillo físico agrícola, RSTP bloqueará automáticamente uno de los enlaces redundantes inter-switches (estado Discarding).
- Edge Ports: Todo puerto de acceso conectado a un ESP32 o a la Raspberry Pi 5 se configurará obligatoriamente como Edge Port. Esto omite las fases de escucha/aprendizaje, permitiendo transmisión inmediata y previniendo que los sensores pierdan conectividad cuando haya un recálculo en la troncal

Enrutamiento (Router-on-a-stick y Salida WAN)

Dado que el tráfico de telemetría de la VLAN 10 necesita llegar al broker MQTT alojado en la VLAN 30, usaremos el modelo de enrutamiento "Router-on-a-stick" centralizado en el router WAN (Teltonika/Cisco)

- Inter-VLAN Routing: El enlace troncal del MikroTik se conectará a un único puerto físico del router. En el router, crearemos subinterfaces lógicas para cada VLAN (eth0.10, eth0.20, eth0.30), configurándoles el Gateway IP de cada subred (10.0.10.1).
- Aislamiento y ACLs: El router permitirá el enrutamiento bidireccional entre la VLAN 10 (Sensores) y la VLAN 30 (Edge), pero se implementarán Listas de Control de Acceso (ACL) para bloquear absolutamente que la VLAN 20 (Cámaras) alcance a la Raspberry Pi 5, blindando la capa de computación de borde [10].

- Salida a la Nube (Cloud): Siguiendo nuestro marco teórico para minimizar el uso de CPU en la granja, el Teltonika usará una simple ruta estática por defecto hacia la antena Starlink o ISP rural, canalizando a la nube únicamente las alertas filtradas por el procesamiento en el borde.

Configuración CCL

Configuración del Router Cisco 7200 (Gateway WAN / Router-on-a-Stick)

El Router Cisco centraliza el enrutamiento Inter-VLAN (Router-on-a-Stick), provee servicios de asignación dinámica de IPs (DHCP) y habilita la salida a Internet a los nodos a través de NAT/PAT.

```
enable

configure terminal

hostname RouterGateway

interface FastEthernet0/0

    no ip address

    no shutdown

exit

interface FastEthernet0/0.10

    encapsulation dot1Q 10

    ip address 10.0.10.1 255.255.254.0

    ip nat inside

exit

interface FastEthernet0/0.20

    encapsulation dot1Q 20

    ip address 10.0.20.1 255.255.255.0

    ip nat inside

exit

interface FastEthernet0/0.30

    encapsulation dot1Q 30
```

```

ip address 10.0.30.1 255.255.255.0

ip nat inside

exit

interface FastEthernet0/0.99

encapsulation dot1Q 99

ip address 10.0.99.1 255.255.255.252

exit

ip dhcp excluded-address 10.0.10.1

ip dhcp pool VLAN10_IOT

network 10.0.10.0 255.255.254.0

default-router 10.0.10.1

dns-server 8.8.8.8

ip dhcp excluded-address 10.0.20.1

ip dhcp pool VLAN20_CCTV

network 10.0.20.0 255.255.255.0

default-router 10.0.20.1

dns-server 8.8.8.8

interface FastEthernet1/0

ip address dhcp

ip nat outside

no shutdown

exit

access-list 1 permit 10.0.0.0
0.255.255.255

ip nat inside source list 1 interface
FastEthernet1/0 overload

no ip nat service alg udp dns

no ip nat service alg tcp dns

access-list 100 deny ip 10.0.10.0
0.0.1.255 10.0.20.0 0.0.0.255

access-list 100 permit ip any any

```

```

interface FastEthernet0/0.10

ip access-group 100 in

exit

end

write memory

```

Configuración del Switch MikroTik CRS328

Los switches MikroTik actúan como concentradores de capa 2. Reciben el enlace troncal (Trunk) desde el router y entregan los accesos desempaquetados a los nodos finales en sus respectivas VLANs.

Script para SW-Dis

```

/interface bridge add
name=bridge_agricola protocol-mode=rstp
priority=4096 vlan-filtering=no

/interface bridge port add
bridge=bridge_agricola interface=ether13
pvid=30 edge=yes

/interface bridge port add
bridge=bridge_agricola interface=ether14
pvid=30 edge=yes

/interface bridge port add
bridge=bridge_agricola interface=ether15

/interface bridge port add
bridge=bridge_agricola interface=ether16

/interface bridge port add
bridge=bridge_agricola interface=ether21

/interface bridge port add
bridge=bridge_agricola interface=ether22

/interface bridge vlan add
bridge=bridge_agricola
tagged=ether15,ether16,ether21,ether22
vlan-ids=10

/interface bridge vlan add
bridge=bridge_agricola
tagged=ether15,ether16,ether21,ether22
vlan-ids=20

/interface bridge vlan add
bridge=bridge_agricola

```

```

tagged=ether15,ether16,ether21,ether22
untagged=ether13,ether14 vlan-ids=30

/interface bridge vlan add
bridge=bridge_agricola
tagged=ether15,ether16,ether21,ether22
vlan-ids=99

/interface bridge set bridge_agricola
vlan-filtering=yes

```

Script para SW-ACSS-A (Igual para SW-Acc-B cambiando ether21 por ether22):

```

/interface bridge add
name=bridge_agricola protocol-mode=rstp
priority=8192 vlan-filtering=no

/interface bridge port add
bridge=bridge_agricola interface=ether1
pvid=10 edge=yes

/interface bridge port add
bridge=bridge_agricola interface=ether2
pvid=10 edge=yes

/interface bridge port add
bridge=bridge_agricola interface=ether3
pvid=10 edge=yes

/interface bridge port add
bridge=bridge_agricola interface=ether4
pvid=10 edge=yes

/interface bridge port add
bridge=bridge_agricola interface=ether5
pvid=10 edge=yes

/interface bridge port add
bridge=bridge_agricola interface=ether6
pvid=10 edge=yes

/interface bridge port add
bridge=bridge_agricola interface=ether7
pvid=10 edge=yes

/interface bridge port add
bridge=bridge_agricola interface=ether8
pvid=10 edge=yes

/interface bridge port add
bridge=bridge_agricola interface=ether9
pvid=20 edge=yes

```

```

/interface bridge port add
bridge=bridge_agricola interface=ether10
pvid=20 edge=yes

/interface bridge port add
bridge=bridge_agricola interface=ether11
pvid=20 edge=yes

/interface bridge port add
bridge=bridge_agricola interface=ether12
pvid=20 edge=yes

/interface bridge port add
bridge=bridge_agricola interface=ether21

/interface bridge port add
bridge=bridge_agricola interface=ether23

/interface bridge vlan add
bridge=bridge_agricola
tagged=ether21,ether23
untagged=ether1,ether2,ether3,ether4,ether5,ether6,ether7,ether8 vlan-ids=10

/interface bridge vlan add
bridge=bridge_agricola
tagged=ether21,ether23
untagged=ether9,ether10,ether11,ether12
vlan-ids=20

/interface bridge vlan add
bridge=bridge_agricola
tagged=ether21,ether23 vlan-ids=30

/interface bridge vlan add
bridge=bridge_agricola
tagged=ether21,ether23 vlan-ids=99

/interface bridge set bridge_agricola
vlan-filtering=yes

```

Subida y Compilación de los Dockers

1. Subimos las carpetas de los proyectos (Desde PowerShell Windows)

```

scp -r
"c:\Users\USER1\GNS3\projects\redes_agricultura_iot\docker_esp32"
gns3@192.168.1.121:/home/gns3/

scp -r
"c:\Users\USER1\GNS3\projects\redes

```

```
_agricultura_iot\docker_edge"
gns3@192.168.1.121:/home/gns3/
```

2. Compilamos las imagenes en GNS3 VM (Desde PowerShell Windows)

```
ssh gns3@192.168.1.121

cd /home/gns3/docker_esp32 &&
docker build -t esp32_agricola:v1 .

cd /home/gns3/docker_edge && docker
build -t edge_agricola:v1 .

exit
```

Configuración de Red e inicializacion del Nodo Edge (Receptor - VLAN 30)

1. Configuración de IP

```
auto eth0

iface eth0 inet static

    address 10.0.30.5

    netmask 255.255.255.0

    gateway 10.0.30.1

    up echo "nameserver 1.1.1.1" >
/etc/resolv.conf
```

2. Ejecución de Simulación

```
mosquitto -c
/etc/mosquitto/mosquitto.conf -d

mosquitto_sub -h localhost -t
"estaciones/#" -v
```

Configuración del Nodo IoT (Transmisor ESP32 - VLAN 10)

1. Configuración de IP (Para cada ESP32)

```
auto eth0

iface eth0 inet static

    address 10.0.10.X
```

```
netmask 255.255.254.0
```

```
gateway 10.0.10.1
```

```
up echo "nameserver 1.1.1.1" >
/etc/resolv.conf
```

2. Ejecución de Simulación

```
python3 /simulador_esp32.py
```

Script Python (Docker) para emular el Gateway ESP32 (MQTT)

Para probar que el enrutamiento y las VLANs funcionan, se abre la consola del contenedor Docker que simula al ESP32 (la cual debe tener la IP asignada 10.0.10.5 y tener la librería Paho-MQTT instalada.

El siguiente script en Python emula la lógica C++ del ESP32, generando lecturas y publicándolas en los tópicos de la Raspberry Pi (10.0.30.5).

```
import paho.mqtt.client as mqtt
```

```
import time
```

```
import random
```

```
import socket
```

```
MQTT_BROKER = "10.0.30.5"
```

```
MQTT_PORT = 1883
```

```
hostname = socket.gethostname()
```

```
TOPIC_TEMP =
f"estaciones/{hostname}/temperatura"
```

```
TOPIC_HUM =
f"estaciones/{hostname}/humedad"
```

```
TOPIC_SUELO =
f"estaciones/{hostname}/humedad_Suelo"
```

```
def on_connect(client, userdata, flags,
rc):
```

```
    if rc == 0:
```

```

        print(f"[+] Conectado al broker
MQTT en 10.0.30.5 como {hostname}")

```

```

    else:

```

```

        print(f"[-] Falla de conexion.
Codigo: {rc}")

```

```

client
=
mqtt.Client(mqtt.CallbackAPIVersion.VERSION1, f"ESP32_{hostname}")

```

```

client.on_connect = on_connect

```

```

print(f"Conectando a {MQTT_BROKER}...")

```

```

try:

```

```

        client.connect(MQTT_BROKER,
MQTT_PORT, 60)

```

```

except Exception as e:

```

```

    print(f"[-] Error: {e}")

```

```

    exit(1)

```

```

client.loop_start()

```

```

try:

```

```

    while True:

```

```

        temperatura
=
round(random.uniform(15.0, 35.0), 2)

```

```

        humedad_rel
=
round(random.uniform(40.0, 80.0), 2)

```

```

        humedad_suelo
=
round(random.uniform(20.0, 60.0), 2)

```

```

        client.publish(TOPIC_TEMP,
str(temperatura), qos=0)

```

```

        client.publish(TOPIC_HUM,
str(humedad_rel), qos=0)

```

```

        client.publish(TOPIC_SUELO,
str(humedad_suelo), qos=0)

```

```

        print(f"Enviado -> Temp:
{temperatura}°C | Hum: {humedad_rel}% |
Suelo: {humedad_suelo}%")

```

```

        time.sleep(5)

```

```

except KeyboardInterrupt:

```

```

    client.loop_stop()

```

```

    client.disconnect()

```

Evidencia de Simulación Base

Topología en el Simulador

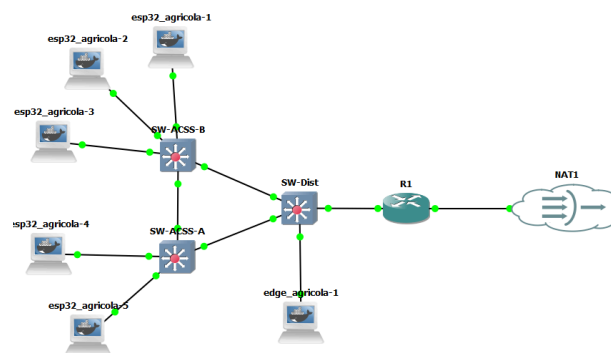


Figura 1. Topología Base de la Red simulada en GNS3 usando dockers, 3 switches y 1 router.

```

[+] Conectado exitosamente al Broker MQTT en el Edge Node (10.0.30.5)
Publicado -> Temp: 20.88°C | Hum: 40.46% | Suelo: 35.32%
[+] Conectado exitosamente al Broker MQTT en el Edge Node (10.0.30.5)
[+] Conectado exitosamente al Broker MQTT en el Edge Node (10.0.30.5)
[+] Conectado exitosamente al Broker MQTT en el Edge Node (10.0.30.5)
[+] Conectado exitosamente al Broker MQTT en el Edge Node (10.0.30.5)
Publicado -> Temp: 27.33°C | Hum: 68.24% | Suelo: 47.16%
[+] Conectado exitosamente al Broker MQTT en el Edge Node (10.0.30.5)
[+] Conectado exitosamente al Broker MQTT en el Edge Node (10.0.30.5)
[+] Conectado exitosamente al Broker MQTT en el Edge Node (10.0.30.5)
Publicado -> Temp: 18.99°C | Hum: 74.16% | Suelo: 43.26%
[+] Conectado exitosamente al Broker MQTT en el Edge Node (10.0.30.5)
[+] Conectado exitosamente al Broker MQTT en el Edge Node (10.0.30.5)
[+] Conectado exitosamente al Broker MQTT en el Edge Node (10.0.30.5)
Publicado -> Temp: 31.46°C | Hum: 76.81% | Suelo: 51.82%
[+] Conectado exitosamente al Broker MQTT en el Edge Node (10.0.30.5)
[+] Conectado exitosamente al Broker MQTT en el Edge Node (10.0.30.5)
[+] Conectado exitosamente al Broker MQTT en el Edge Node (10.0.30.5)
Publicado -> Temp: 23.0°C | Hum: 46.48% | Suelo: 23.92%
[+] Conectado exitosamente al Broker MQTT en el Edge Node (10.0.30.5)
[+] Conectado exitosamente al Broker MQTT en el Edge Node (10.0.30.5)

```

Figura 2. Compilación del código simulacion_esp32 en el docker IoT para simular envío de paquetes MQTT

```

estacion1/humedad_Suelo 35.6
estacion1/temperatura 34.9
estacion1/temperatura 16.02
estacion1/humedad 61.21
estacion1/humedad_Suelo 56.03
estacion1/temperatura 21.37
estacion1/humedad 62.18
estacion1/humedad_Suelo 29.86
estacion1/temperatura 24.06
estacion1/humedad 68.02
estacion1/humedad_Suelo 36.52
estacion1/temperatura 21.71
estacion1/humedad 72.95
estacion1/humedad_Suelo 23.47
estacion1/temperatura 23.51
estacion1/humedad 50.3
estacion1/humedad_Suelo 48.52
estacion1/temperatura 32.96
estacion1/humedad 76.3
estacion1/humedad_Suelo 58.33
estacion1/temperatura 25.29
estacion1/humedad 49.92
estacion1/humedad_Suelo 51.87

```

Figura 3. Lectura de paquetes en el servidor Edge.

VLANs e Inter-VLAN funcionando

```

RouterGateway#show ip route connected
C 192.168.42.0/24 is directly connected, FastEthernet1/0
C 10.0.0.0/8 is variably subnetted, 4 subnets, 3 masks
C 10.0.10.0/23 is directly connected, FastEthernet0/0.10
C 10.0.30.0/24 is directly connected, FastEthernet0/0.30
C 10.0.20.0/24 is directly connected, FastEthernet0/0.20
C 10.0.99.0/30 is directly connected, FastEthernet0/0.99
RouterGateway#

```

Figura 4. VLANs e Inter-VLAN funcionales visualizados en el RouterGateway

Comprobación STP

Esta captura de la consola del SW-Dist demuestra que el switch central ha asumido exitosamente el rol de Root Bridge (root-bridge: yes) en la topología. El identificador del puente raíz (root-bridge-id) tiene el prefijo hexadecimal 0x1000 que equivale a la prioridad decimal 4096 que configuramos manualmente. Al ser el nodo central y la raíz del árbol STP, su campo root-port figura en none (no tiene puerto raíz), ya que todo el tráfico de la red converge hacia él.

```

[admin@MikroTik] > /interface bridge monitor bridge_agricola
state: enabled
current-mac-address: 0C:5E:62:40:00:00
root-bridge: yes
root-bridge-id: 0x1000.0C:5E:62:40:00:00
root-path-cost: 0
root-port: none
port-count: 19
designated-port-count: 4
fast-forward: no
[Q quit][D dump][C-z pause]

```

Figura 5. Visualización del monitor de SW-Dist indicando que es el principal

Esta tabla muestra el estado de los puertos del switch de acceso. Se observa que:

- Los puertos ether2 y ether3 están activos (sin la bandera I de inactivos) y pertenecen a la VLAN 10 (PVID 10) de los sensores, cursando la telemetría en tiempo real.
- Los puertos troncales ether22 (hacia el Core) y ether23 (redundancia hacia el otro switch de acceso) están activos y en el bridge bajo la VLAN 1 por defecto para el paso de tramas BPDUs de Spanning Tree.
- El resto de los puertos físicos (ether4 al ether12) se encuentran correctamente en estado inactivo (I - INACTIVE), lo que incrementa la seguridad de la red al evitar puertos de acceso huérfanos encendidos.

```

[admin@MikroTik] > /interface bridge port print
Flags: I - INACTIVE
Columns: INTERFACE, BRIDGE, HW, PVID, PRIORITY, PATH-COST, INTERNAL-PATH-COST, HORIZON
# INTERFACE BRIDGE HW PVID PRIORITY PATH-COST IN HORIZON
0 ether2 bridge_agricola yes 10 0x80 10 10 none
1 ether3 bridge_agricola yes 10 0x80 10 10 none
2 I ether4 bridge_agricola yes 10 0x80 10 10 none
3 I ether5 bridge_agricola yes 10 0x80 10 10 none
4 I ether6 bridge_agricola yes 10 0x80 10 10 none
5 I ether7 bridge_agricola yes 10 0x80 10 10 none
6 I ether8 bridge_agricola yes 10 0x80 10 10 none
7 I ether9 bridge_agricola yes 20 0x80 10 10 none
8 I ether10 bridge_agricola yes 20 0x80 10 10 none
9 I ether11 bridge_agricola yes 20 0x80 10 10 none
10 I ether12 bridge_agricola yes 20 0x80 10 10 none
11 ether22 bridge_agricola yes 1 0x80 10 10 none
12 ether23 bridge_agricola yes 1 0x80 10 10 none

```

Figura6. Estado de Puertos en el Switch de Acceso con RSTP y VLANs.

Captura de Tráfico

Figura 7. Captura del tráfico de paquetes STP en Wireshark.

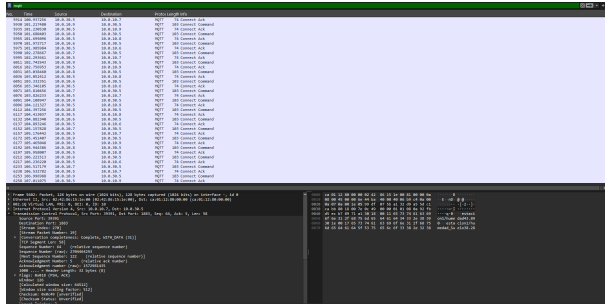


Figura 8. Captura del tráfico de paquetes MQTT en Wireshark.

Evaluación de Métricas Iniciales

Tabla de Tiempos Ping (Latencia Inter-VLAN)

Origen	Destino	Latencia mínima (ms)	Latencia Promedio (ms)
ESP32 (VLAN10)	Gateway (10.0.10.1)	3.191	10.917
ESP32 (VLAN10)	Edge (10.0.30.1)	18.942	34.586
Edge (VLAN30)	Gateway (10.0.10.1)	3.250	11.030

Prueba de Convergencia STP (Simulación de Fallo)

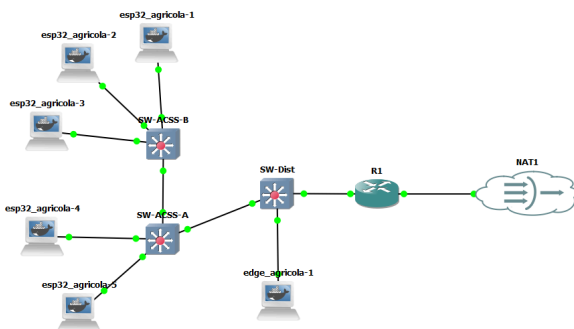


Figura 9. Eliminación de la conexión física eth22 del SW-ACSS y eth22 del SW-Dist para simular un fallo

Se evidencia luego de hacer la captura de tráfico, la correcta operación del protocolo RSTP ante la caída del enlace principal, logrando la restauración del servicio en aproximadamente 2 segundos. La secuencia técnica del evento es la siguiente:

- Interrupción del Enlace Principal: Se registra una pérdida temporal de conectividad. Las solicitudes ICMP Echo enviadas por el host 10.0.10.6 (secuencias 197 a 201) no obtienen respuesta.
- Recálculo de Topología (RSTP): El sistema transmite paquetes STP hacia la dirección multicast. El Root Bridge (Root = 4096) ordena el recálculo de las rutas, lo que transiciona el puerto redundante (ether23) de estado bloqueado a estado de reenvío.
- Actualización de Tablas ARP: Al abrirse la ruta alternativa, los nodos resuelven de inmediato la nueva topología física mediante el protocolo ARP (solicitud y respuesta para la IP 10.0.30.1).
- Restablecimiento del Tráfico: La comunicación se recupera de forma exitosa a partir de la secuencia ICMP 202, reanudando el flujo bidireccional de paquetes.

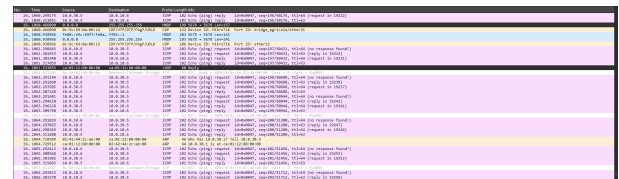


Figura 10. Captura del tráfico cuando ocurre un fallo

Throughput Básico

Medición de ancho de banda máximo utilizando iperf3 en una ventana de 10 segundos:

- Comando Servidor (Edge Node): `iperf3 -s`
- Comando Cliente (ESP32): `iperf3 -c 10.0.30.5 -t 10`

Resultados obtenidos:

- Throughput del Emisor (Emisor): 1.36 Mbits/sec (Transferido: 1.62 MBytes)
- Throughput del Receptor (Receptor): 1.03 Mbits/sec (Transferido: 1.25 MBytes)
- Retransmisiones (Retr): 47

Justificación:

- Cuello de Botella de Emulación originado porque el Router Cisco c7200 corre bajo Dynamips, que emula la CPU MIPS del router

utilizando la CPU física de la computadora host a través de software. Esto limita drásticamente la capacidad de procesar paquetes a altas velocidades.

- Los tres switches MikroTik CHR se ejecutan en máquinas virtuales QEMU, agregando retraso de procesamiento y sobrecarga al encapsular el tráfico de las VLANs.
- Retransmisiones y Pérdida de Paquetes, pues como se observó, la métrica muestra 47 retransmisiones (Retr). Cuando la CPU del router se satura procesando el tráfico de iperf3 (que exige el máximo rendimiento), se descartan paquetes en las colas de las interfaces virtuales, lo que fuerza a TCP a retransmitir y a bajar el ancho de banda efectivo.

```
/ # iperf3 -c 10.0.30.5 -t 10
Connecting to host 10.0.30.5, port 5201
[ 5] local 10.0.10.8 port 47118 connected to 10.0.30.5 port 5201
[ ID] Interval      Transfer    Bitrate      Retr  Cwnd
[ 5] 0.00-1.00 sec   512 KBytes  4.19 Mbits/sec  24   48.1 KBytes
[ 5] 1.00-2.00 sec   256 KBytes  2.10 Mbits/sec  14   24.0 KBytes
[ 5] 2.00-3.00 sec   128 KBytes  1.05 Mbits/sec   0   28.3 KBytes
[ 5] 3.00-4.00 sec    0.00 Bytes  0.00 bits/sec   5   14.1 KBytes
[ 5] 4.00-5.00 sec   256 KBytes  2.10 Mbits/sec   0   19.8 KBytes
[ 5] 5.00-6.00 sec   128 KBytes  1.05 Mbits/sec   0   22.6 KBytes
[ 5] 6.00-7.00 sec    0.00 Bytes  0.00 bits/sec   2   21.2 KBytes
[ 5] 7.00-8.00 sec   256 KBytes  2.10 Mbits/sec   0   24.0 KBytes
[ 5] 8.00-9.00 sec    0.00 Bytes  0.00 bits/sec   2   17.0 KBytes
[ 5] 9.00-10.00 sec  128 KBytes  1.05 Mbits/sec   0   24.0 KBytes
-----
[ ID] Interval      Transfer    Bitrate      Retr
[ 5] 0.00-10.00 sec  1.62 MBytes  1.36 Mbits/sec  47
[ 5] 0.00-10.17 sec  1.25 MBytes  1.03 Mbits/sec
sender
receiver
```

Figura 11. Ejecución del comando para medir el ancho de banda máximo usando iperf3 en Emisor

```
/ # iperf3 -s
Server listening on 5201 (test #1)
Accepted connection from 10.0.10.8, port 47102
[ 5] local 10.0.30.5 port 5201 connected to 10.0.10.8 port 47118
[ ID] Interval      Transfer    Bitrate
[ 5] 0.00-1.00 sec   256 KBytes  2.09 Mbits/sec
[ 5] 1.00-2.00 sec   128 KBytes  1.05 Mbits/sec
[ 5] 2.00-3.00 sec   128 KBytes  1.05 Mbits/sec
[ 5] 3.00-4.00 sec   128 KBytes  1.05 Mbits/sec
[ 5] 4.00-5.00 sec    0.00 Bytes  0.00 bits/sec
[ 5] 5.00-6.00 sec   128 KBytes  1.05 Mbits/sec
[ 5] 6.00-7.00 sec   128 KBytes  1.05 Mbits/sec
[ 5] 7.00-8.00 sec   128 KBytes  1.05 Mbits/sec
[ 5] 8.00-9.00 sec   128 KBytes  1.05 Mbits/sec
[ 5] 9.00-10.00 sec  128 KBytes  1.05 Mbits/sec
[ 5] 10.00-10.17 sec  0.00 Bytes  0.00 bits/sec
-----
[ ID] Interval      Transfer    Bitrate
[ 5] 0.00-10.17 sec  1.25 MBytes  1.03 Mbits/sec
receiver
Server listening on 5201 (test #2)
```

Figura 12. Ejecución del comando para medir el ancho de banda máximo usando iperf3 en Receptor

Referencias

- [1] J. Arboleda Huamán y C. Rodríguez Rodríguez, "Internet de las Cosas en la Agricultura: Estado Actual y Perspectivas Futuras," *Revista Peruana de Ciencias de la Salud*, vol. 7, no. 2, pp. 111-125, 2025, doi: 10.15381/rpcs.v7i2.31324.
- [2] D. Pérez y R. Risco, "Implementación de LoRa y LoRaWAN como escenario futuro de la Industria 4.0 en el sector agroindustrial peruano," Universidad de San Martín de Porres, 2020. [En línea]. Disponible: https://www.usmp.edu.pe/campus/pdf/articulos/articulo_20.pdf
- [3] D. Sernández-Iglesias et al., "Internet of Things Platform for Assessment and Research on Cybersecurity of Smart Rural Environments," *Future Internet*, vol. 17, no. 8, p. 351, Aug. 2025, doi: 10.3390/fi17080351.
- [4] J. P. Rodríguez, A. I. Montoya-Munoz, C. Rodriguez-Pabon, J. Hoyos, y J. C. Corrales, "IoT-Agro: A smart farming system to Colombian coffee farms," *Computers and Electronics in Agriculture*, vol. 190, p. 106442, Nov. 2021, doi: 10.1016/j.compag.2021.106442.
- [5] C. Shewale et al., "Edge AI for Real-Time Agricultural Monitoring Using IoT," *International Journal of Creative Research Thoughts*, vol. 14, no. 4, pp. 1463-1472, Abr. 2026.
- [6] P. Yu, F. Teng, W. Zhu, C. Shen, Z. Chen, y J. Song, "Cloud-edge-device collaborative computing in smart agriculture: architectures, applications, and future perspectives," *Front. Plant Sci.*, vol. 16, p. 1668545, 2025, doi: 10.3389/fpls.2025.1668545.
- [7] A. Francis, U. Stanley, E. O. N. Enehizena, V. M. Olu, A. Peter, y N. Nkordeh, "Design and Analysis of a Broadcast Network Using Logical Segmentation," *TELKOMNIKA Telecommunication, Computing, Electronics and Control*, vol. 16, no. 2, pp. 803-810, Abr. 2018, doi: 10.12928/TELKOMNIKA.v16i2.8461.
- [8] M. A. Romero Portilla, "Diseño de un sistema de monitoreo y gestión de humedad en el suelo de cultivo de papa en la finca Villa Lola, utilizando una red de sensores inalámbricos (WSN) y tecnologías LPWAN," Tesis de grado, Fac. Ing. Cienc. Apl., Univ. Técnica del Norte, Ibarra, Ecuador, 2024.
- [9] L. A. Mostacero Agama, "Análisis de la latencia del servicio de Internet y su impacto en las aplicaciones de Internet de las Cosas (IoT)," Tesis de grado, Fac. Ing., Univ. San Ignacio de Loyola, Lima, Perú, 2022.
- [10] T. Demir, K. Tanko, y S. Hajrulla, "Vlan Implementation Using Numerical Modeling," *International Journal of Advanced Natural Sciences and Engineering Researches*, vol. 9, no. 8, pp. 193-201, 2025.
- [11] Cisco, "Spanning Tree Protocol Problems and Related Design Considerations," Cisco Support, 2024. [En línea]. Disponible: <https://www.cisco.com/c/en/us/support/docs/lan-switching/spanning-tree-protocol/24062-146.html>
- [12] Cisco Community, "Spanning Tree Protocol's Inefficiency in Ring Topologies," Cisco Networking Blogs, 2025. [En línea]. Disponible: <https://community.cisco.com/t5/networking-blogs/spanning-tree-protocols-inefficiency-in-ring-topologies/ba-p/5203662>
- [13] I. O. Adewumi, A. O. D. Adejumo, O. Adegboakan, y E. O. Ogundare, "Edge AI-IoT Integration for Real-Time Precision Farming: A Framework for Adaptive Monitoring, Prediction, and Resource Optimization," *London J. Phys.*, vol. 3, no. 1, pp. 1-12, Mar. 2026. doi: 10.69710/ljp.v3i1.16313.
- [14] J. P. Tovar-Soto, C. F. Pareja-Figueroa, O. L. García-Navarrete, y L. C. Gutiérrez-Martínez, "Performance evaluation of LoRa technology for implementation in rural areas," *DYNA*, vol. 88, no. 216, pp. 71-78, Ene.-Mar. 2021, doi: 10.15446/dyna.v88n216.88299.
- [15] H. V. Mora Magallanes y J. L. Rosas Pari, "Diseño, desarrollo e implementación de una red de sensores inalámbricos (WSN) para el control, monitoreo y toma de decisiones aplicado en la agricultura de precisión basado en Internet de las Cosas (IoT). Caso de estudio cultivo de frijol," Tesis de título profesional, Escuela Profesional de Ingeniería Electrónica, Universidad Ricardo Palma, Lima, Perú, 2019.

[16] Cisco Networking Academy, “Rapid PVST+,” in *Varieties of Spanning Tree Protocols*. [Online]. Available:

<https://www.iipwr.edu.pl/~kano/course/module4/4.2.3.1/4.2.3.1.html>

[17] NetworkAcademy, “Understanding RSTP Synchronization,” *NetworkAcademy.io*. [En línea]. Disponible en:

<https://www.networkacademy.io/ccna/spanning-tree/understanding-rstp-synchronization>

[18] Convergía, “Agricultura de precisión aplicada en cultivos de palta ubicados en el Fundo Caral,” *Convergía*. [En línea]. Disponible en:

<https://convergía.io/es/casos-de-exito/solutions/agricultura-de-precision-aplicada-en-cultivos-de-palta-ubicados-en-el-fundo-caral/>

[19] H. Fazel, “Spanning Tree Protocols: Inefficiency in Ring Topologies,” Cisco Community, 04 de octubre de 2024. [En línea]. Disponible en: <https://community.cisco.com/t5/networking-blogs/spanning-tree-protocols-inefficiency-in-ring-topologies/ba-p/5203662>. [Accedido: 09-jun-2026].

[20] . P. Raut, “A study of the naming convention used for topics in distributed MQTT brokers’ environment,” Tesis de Maestría, Dept. Comput. Sci. (Intelligent Systems), Trinity College, Univ. of Dublin, Dublín, Irlanda, 2022.

[21] D. Sernández-Iglesias, L. Tobarra, R. Pastor-Vargas, A. Robles-Gómez, P. Vidal-Balboa y J. Sarraipa, “Internet of Things Platform for Assessment and Research on Cybersecurity of Smart Rural Environments,” *Future Internet*, vol. 17, no. 8, p. 351, ago. 2025, doi: 10.3390/fi17080351.